

CircuitMentor

Official Architecture & Workflow Documentation

1. Executive Summary

CircuitMentor is an AI-enhanced electronics learning platform designed to take students from an abstract idea (e.g., “Make a motion-activated alarm”) to a physical, working circuit.

Initially relying heavily on LLMs (Groq/Llama-3) for circuit netlist generation, the system suffered from rate limits, unpredictable hallucinations, and high latency. We successfully pivoted the core architecture: **circuit generation is now 100% deterministic, offline, and instant**, while AI is strictly reserved for teaching, explaining code, and answering student questions.

2. System Architecture

The platform is split into a **React/Next.js Frontend** and a **Python FastAPI Backend**.

```
graph TD
    User([User Prompt]) --> Frontend[Next.js Frontend UI]
    Frontend -- "POST /api/generate-circuit" --> Backend[FastAPI Backend]
    subgraph "Deterministic Backend Engine"
        Backend --> Parser[Prompt Parser & Component Detector]
        Parser --> Builder[Local Circuit Builder]
        Builder --> CodeGen[Code Generator & Static Fallback]
    end
    CodeGen -- "JSON Output" --> Frontend
    subgraph "Frontend Visualization"
        Frontend --> RulesEngine[wiringRulesEngine.ts]
        RulesEngine --> ReactFlow[Circuit Canvas Render]
    end
```

```
subgraph "AI Teaching Mode"
  Frontend -- "POST /api/generate-project-
explanation" --> GroqAPI[Groq API - Llama 3.1 8B]
  GroqAPI --> Explanations[Component & Logic
Explanations]
end
```

3. Core Workflows

A. Circuit & Code Generation (Deterministic)

When a user types a prompt like “soil moisture sensor with buzzer”:

- Prompt Parsing (local_circuit_engine.py):** The backend converts the text into a concept block (Inputs, Logic, Outputs) using advanced keyword mapping (COMPONENT_KEYWORDS).
- Circuit Building:** The engine maps these components to a local PIN_MAP (e.g., Soil Sensor -> A2, Buzzer -> Pin 8), guaranteeing safe, predictable pin assignments.
- EIL Gatekeeper Bypass:** Because the local pins are pre-validated, the system securely bypasses the heavy Electronic Intelligence Layer (EIL) checks, reducing latency to ~10ms.
- Code Assembly:** The backend reads a rock-solid, handwritten C++ template (generated_iot_code.ino) which natively adapts to the detected components and serves it to the frontend.

B. Visual Routing (wiringRulesEngine.ts)

Once the frontend receives the chosen components:

- Auto-Routing:** The wiringRulesEngine.ts maps physical layout coordinates.
- Safety Injectors:** The engine automatically detects needed safety hardware (e.g., placing a 220Ω resistor in series with an LED, or a flyback diode on relays).
- Power Rails:** It dynamically builds a 5V and GND bus on a virtual breadboard, routing all power lines cleanly.

C. AI Explanations (LLM)

Once the circuit is successfully generated, the student clicks “Next: Code & Logic Explanation”:

- The frontend securely calls the Groq API utilizing llama-3.1-8b-instant.
- The AI acts as a patient lab instructor, generating non-robotic, relatable analogies for why the circuit works (e.g., explaining Analog to Digital Conversion for the soil sensor).

4. Technical File Registry

File	Purpose
<code>components.json</code>	The single source of truth for hardware. Defines max voltages, current limits, and logic thresholds.
<code>local_circuit_engine.py</code>	The backend workhorse that replaced AI for rapid keyword-based electronics mapping.
<code>eil_validator.py</code>	Electronics Intelligence Layer. A safety rule engine preventing shorts, floating pins, and power drains.
<code>groq_llm.py</code>	The API integration for Llama 3. Manages rate limits and constructs strictly formatted JSON instructor prompts.
<code>wiringRulesEngine.ts</code>	Frontend auto-router that transforms data into specific ReactFlow visual nodes and bezier edges.
<code>generated_iot_code.ino</code>	Universal C++ firmware providing non-blocking logic and unified abstractions for 30+ sensors.

5. Why This Architecture Wins

- **Instantaneous:** Removing the LLM from the critical path dropped generation time from ~20 seconds to less than **0.1 seconds**.
 - **Zero API Limits:** Circuit design works 100% offline, meaning Groq's daily free-tier limits no longer crash the user's primary workspace.
 - **100% Safe Wiring:** LLMs often hallucinate bad wiring (e.g., tying 5V to an ESP32 GPIO). The local engine forces strict hardware adherence.
 - **Cost Effective:** Costs drop to essentially \$0 per local generation, while still benefiting from AI in the cheaper "chat/explanation" phase.
-

Generated for CircuitMentor Development Team.